

TP Sheet 4 - Mapping

Prepared by: Gonalo Leo

With contributions by: Professor Lus Paulo Reis and Professor Armando Sousa

Exercise 1 - Deterministic occupancy grid mapping

Consider the simulation scenario in the mapW.wbt or obstacles.wbt Webots files, with the usual e-puck robot, equipped with a LiDAR sensor. Remember that these files were generated using the create_map.py script and thus the walls are detectable by the LiDAR but the robot can traverse them and no collisions occur (think of them as “holograms”).

Write a robot controller that gradually builds a map of the robot’s environment. The map is represented as an occupancy grid map (2D numpy array), where each cell represents a squared section of the map, with the following attributes:

- origin: real (x,y) coordinate corresponding to the bottom-left pixel of the grid map.
- dimensions: number of columns and rows (in this order) of the grid map.
- resolution: number of meters that are represented by the length of one pixel of the grid map.

The robot can be controlled using the following keyboard keys:

- w: move in the forwards direction the robot is facing.
- s: move in the backwards direction the robot is facing.
- a: rotate left.
- d: rotate right.
- space: take a LiDAR scan of the environment and update the map (which is also saved as a .png file)

In this exercise, each cell of the grid map may have three possible values:

- 0: the cell is thought to be empty/free
- 1: the cell is thought to be occupied/contains an obstacle
- 0.5: unknown cell

In this exercise, you will fill sections of the DeterministicOccupancyGrid class, which is derived from the abstract base class OccupancyGrid, that represents a generic occupancy grid and leaves up to the derived class to represent the grid and fill it in with its own logic. The base class (in occupancy_grid.py) already has methods to convert real (x,y) coordinate to grid (row,col) coordinates (real_to_grid_coords method) and to check if a grid (row,col) coordinate is within the bounds of a grid map (are_grid_coords_in_bounds method).

To help you, use the .py file with the base code provided in the Github repository and fill out these TODOs (in order):

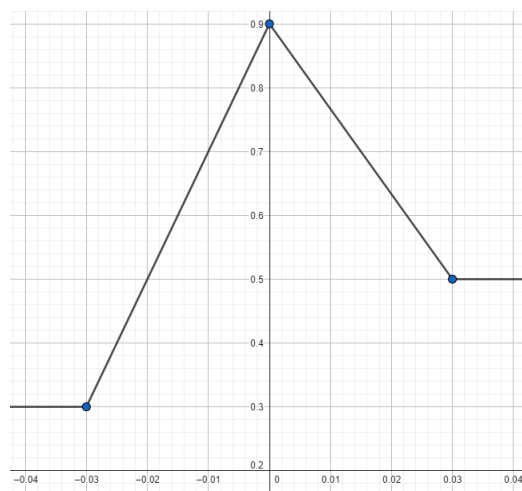
1. In the DeterministicOccupancyGrid class constructor, initialize the occupancy grid map.

2. In the `update_cell` method, fill in the grid's cell according to whether the area is believed to be occupied or free (`is_occupied` attribute).
3. In the `update_map` method,
 - a. Compute the grid (row, col) coordinates of the robot. You can use `get_translation` from `transformations.py` to extract the position from the robot's transformation matrix.
 - b. Compute grid coordinates of each LiDAR scan point.
 - c. Update the grid map by setting robot's current cell as free.
 - d. Update the grid by setting the cells between the robot and each of the LiDAR cells as free. You can use the `bresenham` function from `utils.py` to compute the ordered list of cell coordinates between two cells.
 - e. Update the grid by setting the cells of each LiDAR cell as occupied.
4. Test your solution in different scenarios:
 - a. What happens if you change the map's resolution (and also its dimensions)?
 - b. What if you increase the LiDAR's noise?

Exercise 2 - Probabilistic occupancy grid mapping

You will now work with a second derived class of `OccupancyGrid` named `ProbabilisticOccupancyGrid`, which handles noisy data better. Each cell of the grid represents the probability of the cell being occupied (0.5 still means that the state is unknown). The actual value of each cell is the natural logarithm (i.e. base e) of the probability's odd (called log-odd). Check out the theoretical slides on Mapping for all the formulae on representing and updating probabilistic occupancy grids. The final slide of the slides provides a visual example of the resulting map.

Consider the inverse sensor model depicted below, where the x axis the distance between a cell's real (x,y) coordinate and the LiDAR's measured distance and the y axis is the probability, for which one then applies the log-odd function.



For the special case of the grid cell where the robot is located, consider that the inverse sensor model always returns a fixed probability of 0.1.

To help you, use the .py file with the base code provided in the Github repository and fill out these TODOs (in order):

1. In the ProbabilisticOccupancyGrid class constructor, initialize the occupancy grid map.
2. In the get_log_odd method, fill in the computation of the log-odd of a probability.
3. In the get_probability_grid method, fill in the computation of the 2D numpy array with the occupancy probability of each cell using the class' self.occupancy_grid attribute (without modifying it). This function should perform maximum likelihood mapping and thus round all values to the closest value between 0, 0.5 and 1.0. In other words, this function converts a matrix of log-odds to a matrix of rounded probabilities.
4. In the update_cell method:
 - a. Compute the probabilities of the inverse sensor model for the cells that do not correspond to the robot's current cell (suggestion: use the [interp](#) function from numpy to perform the linear interpolations of the function).
 - b. Update the value of the cell using the Bayesian formula from the theoretical lecture slides.
5. In the update_map method,
 - a. Compute the grid (row, col) coordinates of the robot.
 - b. Update the grid map cell where the robot is located.
 - c. Update the grid by updating the cells on the lines going from the robot up to the corners of the grid map that pass through each LiDAR grid cell.
Note: Unlike the previous exercise, the values of the grid cells behind the obstacles encountered by the LiDAR will also be updated, which will require using an extended version of the Bresenham algorithm. You can use the bresenham_extended function from utils.py to compute the new ordered list.
 - d. Update the grid by setting the cells of each LiDAR cell as occupied.
6. Test your solution in different scenarios:
 - a. What happens if you change the map's resolution (and also its dimensions)?
 - b. What if you increase the LiDAR's noise?